

UNIVERSIDAD AUTÓNOMA “GABRIEL RENE MORENO”

***FACULTAD DE INGENIERÍA Y CIENCIAS DE LA
COMPUTACIÓN Y TELECOMUNICACIONES***



Proyecto Surtidor

Parcial 2

Universitario: Camino Puma Ronald

Registro: 220031797

Materia: Arquitectura de Software

Grupo: SA

Semestre: 1-2025

Docente: Ing. Obed Veizaga

Santa Cruz – Bolivia

Índice

Contenido

1.	Requisitos.....	1
	Funcionales.....	1
	No Funcionales.....	1
	CU2. Gestionar Tipo Combustible.....	2
	CU5. Calcular la Probabilidad de Abastecimiento de combustible.....	4
	CU6. Calcular la Probabilidad de Tiempo para la carga de Combustible.....	5
2.	Análisis.....	6
	2.1. Identificación de Módulos.....	6
	2.1.1. Vista de Modulo Surtidor.....	6
	2.1.2. Vista de Módulo Probabilidad.....	6
3.	Diseño.....	7
	3.1. Diseño de la Arquitectura.....	7
	3.1.1. Módulo de Surtidor.....	7
	3.1.2. Módulo de TipoCombustible.....	7
	3.2. Diseño de la base de datos.....	8
	3.2.1. Diseño Conceptual.....	8
	3.2.2. Diseño Lógico.....	8
	3.2.3. Diseño Físico.....	8
	3.3. Diseño del Detalle Procedimental.....	11
	3.3.1. Patrón Observer (CU 2).....	11
	3.3.2. Patrón Estrategia (CU 5 y 6).....	12

1. Requisitos

Funcionales

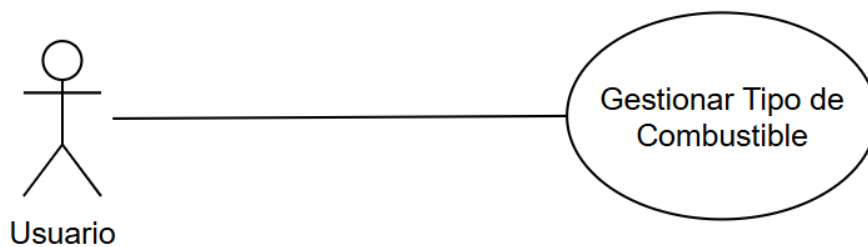
- RF01: El sistema debe permitir registrar nuevos surtidores indicando nombre, coordenadas (latitud y longitud) y cantidad de bombas.
- RF02: El sistema debe mostrar todos los surtidores registrados en un mapa con su respectivo marcador.
- RF03: El sistema debe permitir al usuario seleccionar un surtidor desde un spinner desplegable.
- RF04: El sistema debe obtener y mostrar la ubicación actual del usuario (previo permiso).
- RF05: El sistema debe calcular la distancia entre el usuario y el surtidor seleccionado.
- RF06: El sistema debe calcular el tiempo estimado de espera según la cantidad de autos en fila, la cantidad de bombas y las mangueras disponibles.
- RF07: El sistema debe mostrar una estimación de la probabilidad de abastecimiento considerando la cantidad de combustible disponible, autos en espera y capacidad operativa del surtidor.
- RF08: El sistema debe permitir editar los datos de un surtidor (nombre, ubicación, bombas).
- RF09: El sistema debe permitir eliminar surtidores existentes.
- RF10: El sistema debe almacenar y recuperar todos los datos en una base de datos SQLite interna.

No Funcionales

- RNF01: El sistema debe funcionar en dispositivos Android con versión mínima [especificar, ej: Android 8.0 (API 26)].
- RNF02: El sistema debe tener una interfaz intuitiva y accesible para usuarios sin conocimientos técnicos.
- RNF03: El sistema debe cargar los mapas de forma fluida y sin interrupciones perceptibles para el usuario.

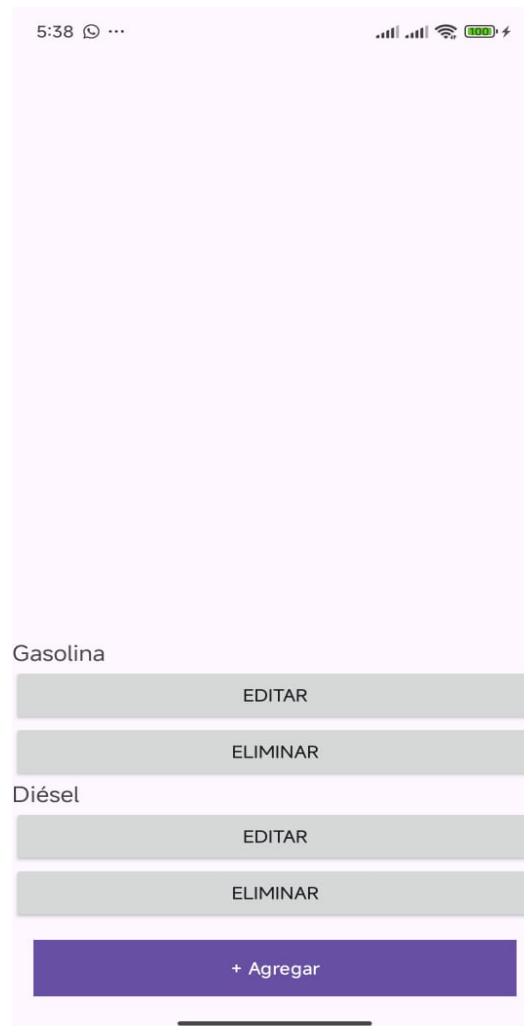
- RNF04: El tiempo de respuesta para cálculos de probabilidad no debe superar los 2 segundos.
- RNF05: La aplicación debe solicitar y respetar los permisos de localización del usuario.
- RNF06: El sistema debe seguir el patrón de arquitectura en 3 capas para separar presentación, lógica de negocio y acceso a datos.
- RNF07: Los datos deben almacenarse localmente sin requerir conexión a internet.
- RNF08: El sistema debe validar entradas del usuario y mostrar mensajes adecuados si hay errores (por ejemplo, campos vacíos o datos inválidos).
- RNF09: El sistema debe estar preparado para escalar, permitiendo agregar nuevas funcionalidades sin afectar las existentes.

CU2. Gestionar Tipo Combustible

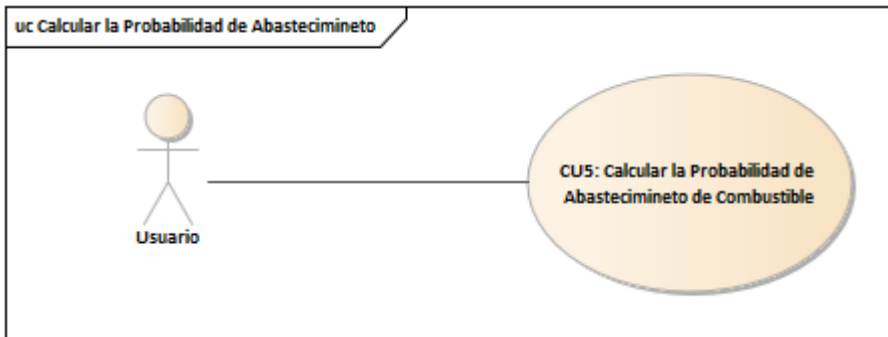


Nombre de Caso de Uso	CU2. Gestionar Tipo Combustible
Propósito	Permite gestionar los tipos de Combustibles
Actores	Usuario
PreCondición	Ninguna
Acciones del Actor	Respuesta del Sistema
1: Crear Tipo de combustible del Surtidor	El sistema guardara nuevo tipo de combustible..

2: Editar Tipo de combustible del Surtidor	El sistema editara nuevo tipo de combustible.
3: Eliminar Tipo de combustible del Surtidor	El Usuario podrá eliminar algún tipo de combustible.
Excepciones	Ninguna.



CU5. Calcular la Probabilidad de Abastecimiento de combustible

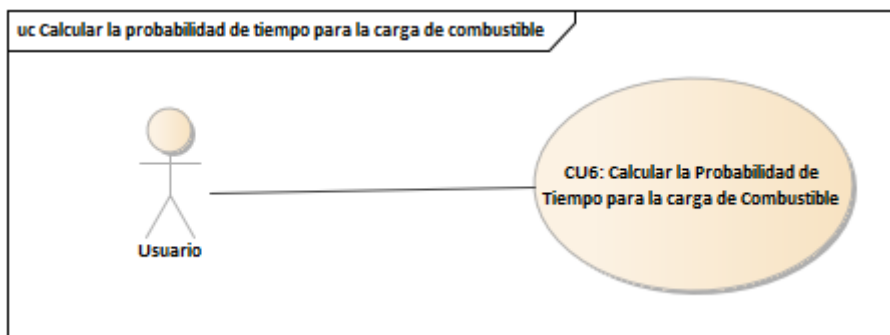


Detalles del caso de Uso

Nombre de Caso de Uso	CU4. Calcular la Probabilidad de Abastecimiento
Propósito	El Usuario podrá saber si vale la pena esperar para que le alcance el combustible al momento de cargar
Actores	Usuario
PreCondición	Tener datos aproximados del Surtidor
Acciones del Actor	Respuesta del Sistema
El usuario podrá insertar datos para realizar los cálculos	El sistema el base a los cálculos insertado por el usuario le mostrará si podrá o no cargar combustible y en que tiempo.
Excepciones	Cargar datos mayores o igual a 0 para los cálculos



CU6. Calcular la Probabilidad de Tiempo para la carga de Combustible



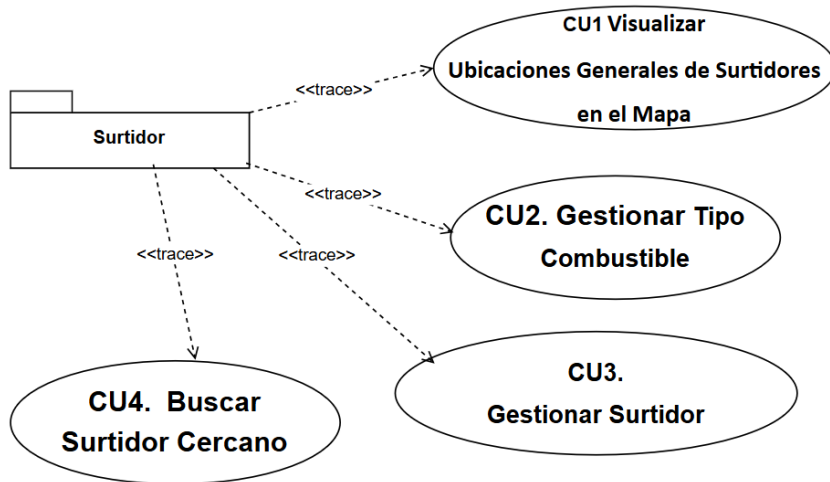
Nombre de Caso de Uso	CU4. Calcular la Probabilidad de Abastecimiento
Propósito	El Usuario podrá saber si vale la pena esperar para que le alcance el combustible al momento de cargar
Actores	Usuario
PreCondición	Tener datos aproximados del Surtidor
Acciones del Actor	Respuesta del Sistema
El usuario podrá insertar datos para realizar los cálculos	El sistema el base a los cálculos insertado por el usuario le mostrará si podrá o no cargar combustible y en que tiempo.
Excepciones	Cargar datos mayores o igual a 0 para los cálculos



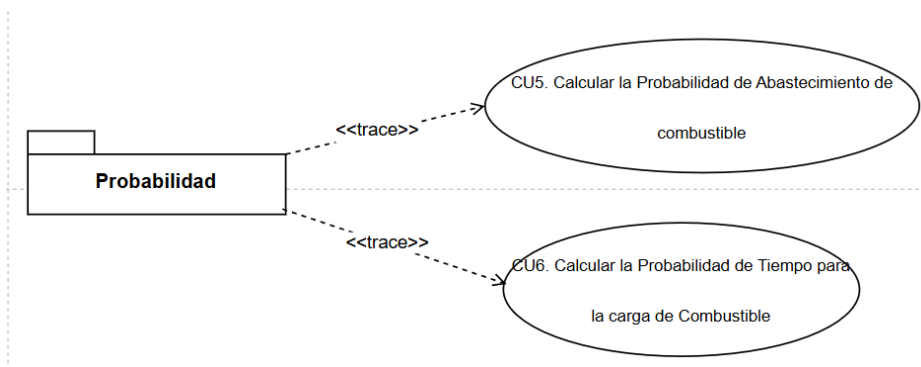
2. Análisis

2.1. Identificación de Módulos

2.1.1. Vista de Módulo Surtidor



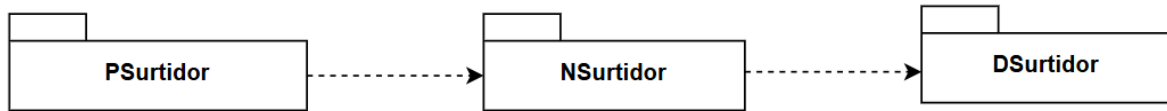
2.1.2. Vista de Módulo Probabilidad



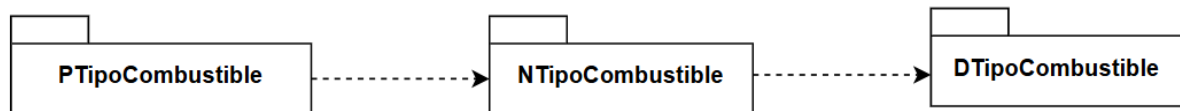
3. Diseño

3.1. Diseño de la Arquitectura

3.1.1. Módulo de Surtidor



3.1.2. Módulo de TipoCombustible

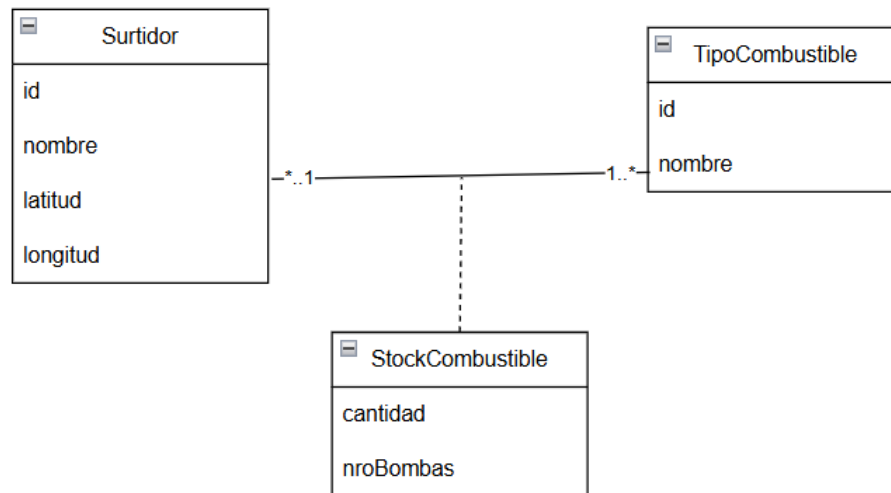


3.1.3. Módulo StockCombustible



3.2. Diseño de la base de datos

3.2.1. Diseño Conceptual



3.2.2. Diseño Lógico

Surtidor						
Pk						
id	nombre	latitud	longitud			
TipoCombustible			stockCombustible			
Pk			PE/FK	PK/FK		
id	nombre		idSurtidor	idTipoCombustible	cantidad	nroBombas

3.2.3. Diseño Físico

```
package com.example.parcialproyectosurtidor.datos.conexion

import android.content.Context
import android.database.Cursor
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class Conexion(context: Context) : SQLiteOpenHelper(context, DATABASE_NAME,
null, DATABASE_VERSION) {

    companion object {
        private const val DATABASE_NAME = "surtidor_db"
        private const val DATABASE_VERSION = 1

        private const val TABLA_SURTIDOR = "Surtidor"
        private const val TABLA_COMBUSTIBLE = "TipoCombustible"
        private const val TABLA_STOCK = "StockCombustible"
    }
}
```

```

// Método que se ejecuta cuando se crea la base de datos
override fun onCreate(db: SQLiteDatabase) {
    // Crear tabla Surtidor
    db.execSQL("""
        CREATE TABLE $TABLA_SURTIDOR (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nombre TEXT NOT NULL,
            latitud REAL NOT NULL,
            longitud REAL NOT NULL
        )
    """)

    // Crear tabla TipoCombustible
    db.execSQL("""
        CREATE TABLE $TABLA_COMBUSTIBLE (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nombre TEXT NOT NULL
        )
    """)

    // Crear tabla StockCombustible
    db.execSQL("""
        CREATE TABLE $TABLA_STOCK (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            id_surtidor INTEGER NOT NULL,
            id_tipo_combustible INTEGER NOT NULL,
            cantidad REAL NOT NULL,
            nroBombas INTEGER NOT NULL,
            FOREIGN KEY (id_surtidor) REFERENCES $TABLA_SURTIDOR(id),
            FOREIGN KEY (id_tipo_combustible) REFERENCES
    $TABLA_COMBUSTIBLE(id)
        )
    """)

    // Insertar tipos de combustible
    db.execSQL("INSERT INTO $TABLA_COMBUSTIBLE (nombre) VALUES ('Gasolina')")
    db.execSQL("INSERT INTO $TABLA_COMBUSTIBLE (nombre) VALUES ('Diésel')")

    // Insertar surtidores
    val surtidores = listOf(
        Triple("Biopetrol-Refineria Oriental", -17.768984, -63.171002),
        Triple("Biopetrol-Beni", -17.769411, -63.178779),
        Triple("Biopetrol-cabezas", -18.787425, -63.314198),
        Triple("Biopetrol-Berrea", -17.837367, -63.238185),
        Triple("Virgen de Cotoca", -17.782294, -63.163588),
        Triple("Biopetrol-Equipetrol", -17.754442, -63.197113),
        Triple("Biopetrol-Gasco", -17.759355, -63.179401),
        Triple("Biopetrol-Teca", -17.764088, -63.071413),
        Triple("Biopetrol-Lopez", -17.725680, -63.165386),
        Triple("Biopetrol-Paragua", -17.764901, -63.149497),
        Triple("Biopetrol-PiraiCamiri", -17.022670, -63.532602),
        Triple("Biopetrol-Pirai", -17.785829, -63.204411),
        Triple("Biopetrol-Royal", -17.805768, -63.197941),
        Triple("Biopetrol-ES Sur", -17.799900, -63.180483),
        Triple("Biopetrol-Viru Viru", -17.675913, -63.159031)
    )
}

```

```

surtidores.forEach { (nombre, lat, lon) ->
    db.execSQL("""
        INSERT INTO $TABLA_SURTIDOR (nombre, latitud, longitud)
        VALUES ('$nombre', $lat, $lon)
    """)
}

// Insertar stock con bombas por tipo de combustible
//1 Gasolina
//2 Diesel
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (1, 2, 10000.0, 4)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (2, 2, 10000.0, 3)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (3, 1, 10000.0, 4)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (4, 1, 10000.0, 2)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (5, 1, 10000.0, 3)")

db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (6, 1, 10000.0, 4)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (7, 1, 10000.0, 3)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (8, 1, 10000.0, 4)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (9, 1, 10000.0, 2)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (10, 1, 10000.0, 3)")

db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (11, 1, 10000.0, 4)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (12, 1, 10000.0, 3)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (13, 1, 10000.0, 4)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (14, 1, 10000.0, 2)")
db.execSQL("INSERT INTO $TABLA_STOCK (id_surtidor,
id_tipo_combustible, cantidad, nroBombas) VALUES (15, 1, 10000.0, 3)")
}

// Método que se ejecuta cuando la versión de la base de datos se
actualiza
override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion:
Int) {
    db.execSQL("DROP TABLE IF EXISTS $TABLA_STOCK")
    db.execSQL("DROP TABLE IF EXISTS $TABLA_COMBUSTIBLE")
    db.execSQL("DROP TABLE IF EXISTS $TABLA_SURTIDOR")
    onCreate(db)
}

// Método para obtener la conexión a la base de datos
fun getConnection(): SQLiteDatabase {
    return this.writableDatabase
}

```

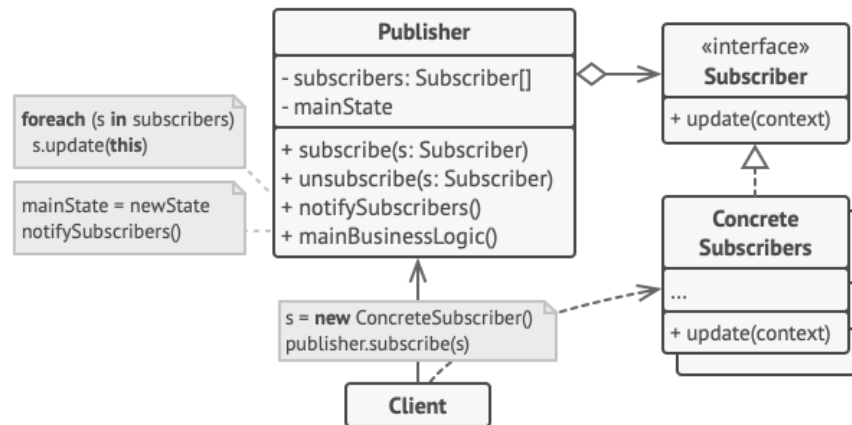
```

    }
}
}

```

3.3.Diseño del Detalle Procedimental

3.3.1. Patrón Observer (CU 2)



CU2: Gestionar Tipo Combustible

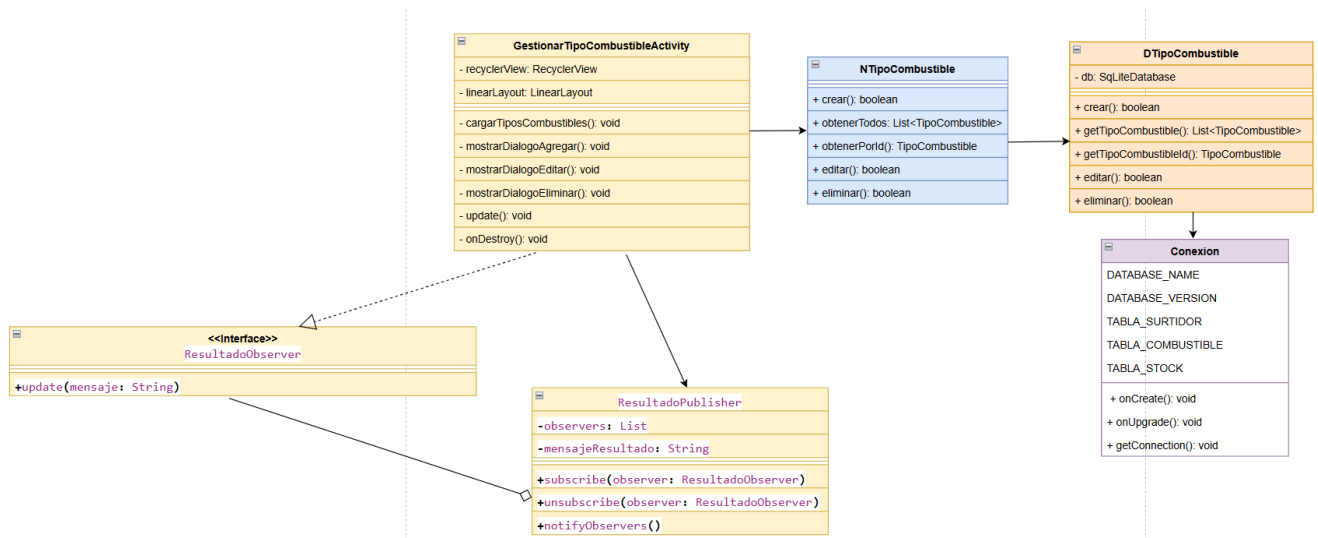
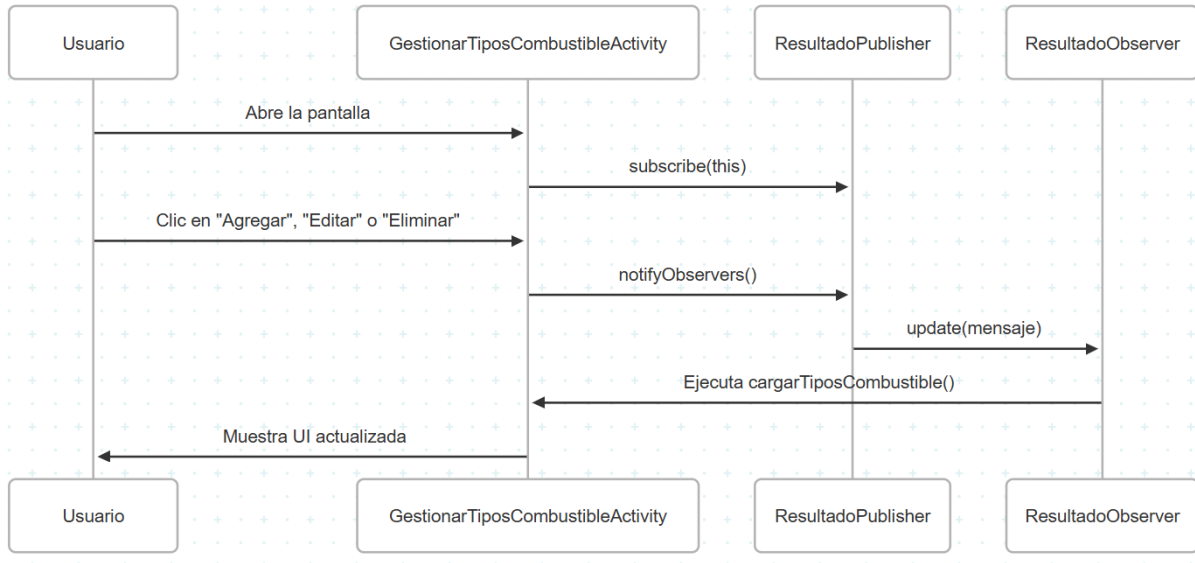
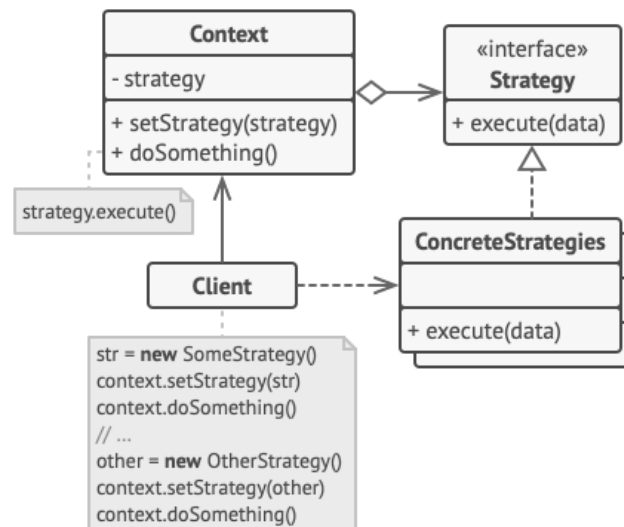


Diagrama de Secuencia



3.3.2. Patrón Estrategia (CU 5 y 6)



CU5: Calcular Probabilidad de Abastecimiento de Combustible

CU6: Calcular Probabilidad de tiempo para la carga de Combustible

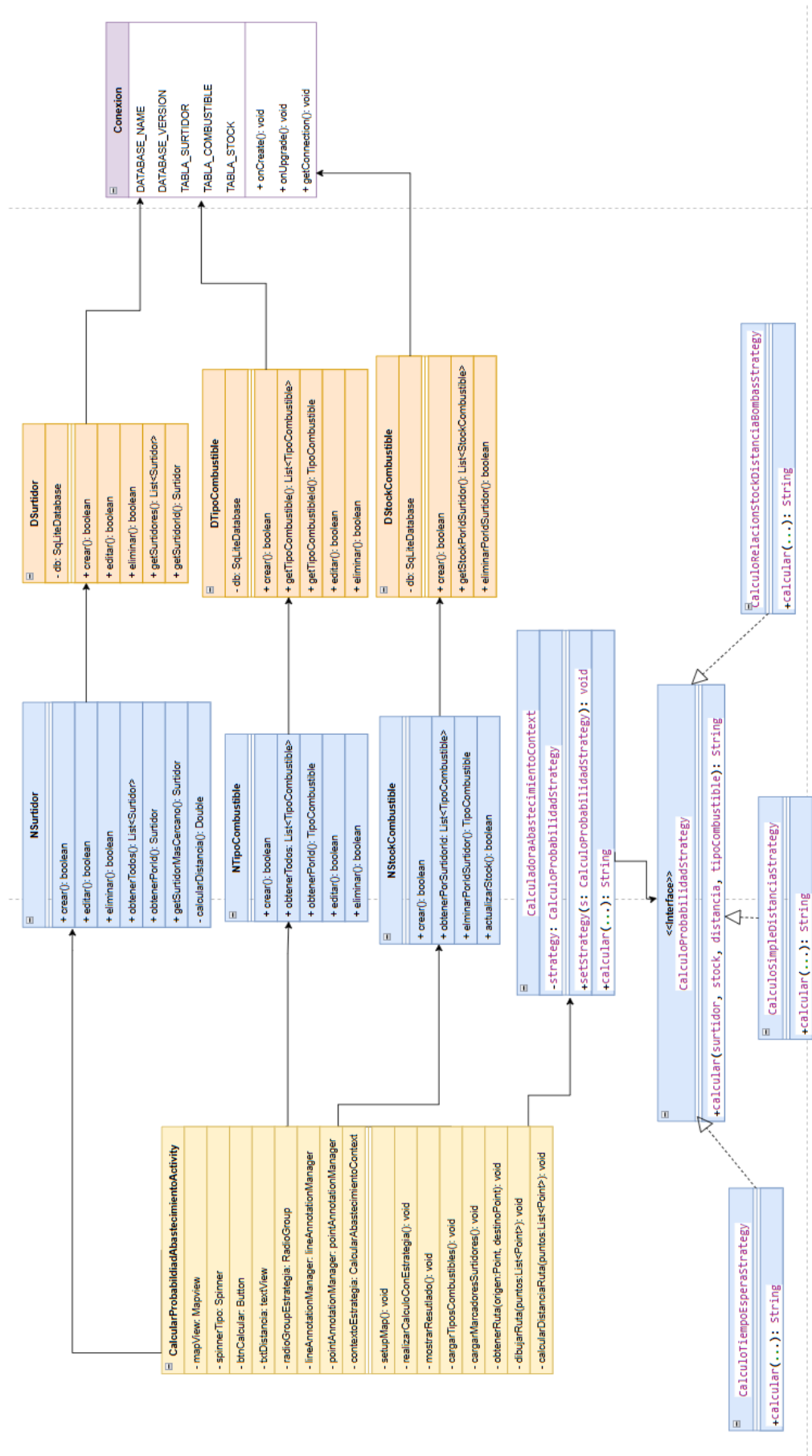


Diagrama de Secuencia

